

Best practices for creating training data

Product is covered by the [Pre-GA Offerings Terms \(/terms/service-terms#1\)](/terms/service-terms#1) of the Google Cloud Platform Terms of Service. Pre-GA products may have limited support, and changes to pre-GA products may not be compatible with other pre-GA versions. For more information, see the [launch stage descriptions \(/products#product-launch-stages\)](/products#product-launch-stages).

This page provides some basic concepts to consider when you are putting together data for an AutoML Tables dataset. It is not meant to be an exhaustive treatment.

Introduction

A well-designed dataset increases the quality of the resulting machine learning model. You can use the guidelines on this page to increase the quality of your dataset and model.

If you are experienced at creating training data for machine learning models, review the list of tasks you do *not* need to worry about. AutoML Tables does many data preparation tasks for you.

Data preparation best practices

Avoid target leakage

Target leakage happens when your training data includes predictive information that is not available when you ask for a prediction. Target leakage can cause your model to show excellent evaluation metrics, but perform poorly on real data.

For example, suppose you want to know how much ice cream your store will sell tomorrow. You cannot include the target day's temperature in your training data, because you will not know the temperature (it hasn't happened yet). However, you could use the *predicted* temperature from the previous day, which could be included in the prediction request.

Avoid training-serving skew

Training-serving skew happens when you generate your training data differently than you generate the data you use to request predictions.

For example, if you use an average value, and for training purposes you average over 10 days, but when you request prediction, you average over the last month.

In general, any difference between how you generate your training data and your serving data (the data you use to generate predictions) should be reviewed to prevent training-serving skew.

Training-serving skew and data distribution

Training-serving skew can also occur based on your data distribution in your training, validation, and testing data splits. There is frequently a difference between the data distribution that a model will see when it is deployed in production versus the data distribution of the dataset that a model is trained on. For example, in production, a model may be applied on an entirely different population of users than those seen during training, or the model may be used to make predictions 30 days after the final training data was recorded.

For best results, ensure that the distribution of the data splits used to create your model accurately reflects the difference between the training data set, and the data that you will be making predictions on in your production environment. AutoML Tables could produce non-monotonic predictions, and if the production data is sampled from a very different distribution than the training data, non-monotonic predictions are not very reliable.

Furthermore, the difference in production data versus training data must be reflected in the difference between the validation data split and the training data split, and between the testing data split and the validation data split.

For example, if you are planning on making predictions about user lifetime value (LTV) over the next 30 days, then make sure that the data in your validation data split is from 30 days after the data in your training data split, and that the data in your testing data split is from 30 days after your validation data split.

Similarly, if you want your model to be tuned to make generalized predictions about new users, then ensure that data from a specific user is only contained in a single split of your training data. For example, all of the rows that pertain to `user1` are in the training data split, all of the rows that pertain to `user2` are in the validation data split, and all of the rows that pertain to `user3` are in the testing data split.

Provide a time signal

If the underlying pattern in your data is likely to shift over time (it is not randomly distributed in time), make sure you provide that information to AutoML Tables. You can provide a time signal in several ways:

- If each row of data has a timestamp, make sure that column is included, has a data type of `Timestamp`, and is set as the **Time** column when you create your dataset. This ordering is used to split the data, with the most recent data as the test data, and the earliest data as the training data. [Learn more](/automl-tables/docs/prepare#time) (/automl-tables/docs/prepare#time).
- If your time column does not have many distinct values, you should use a manual split instead of using the Time column to split your data. Otherwise, you might not get enough rows in each dataset, which can cause training to fail.
- If the time information is not contained in a single column, you can use a [manual data split](/automl-tables/docs/prepare#split) (/automl-tables/docs/prepare#split) to use the most recent data as the test data, and the earliest data as the training data.

Make information explicit where needed

Usually, you do not need to perform feature engineering when you create a model using AutoML Tables. However, for certain data primitives, you can improve model quality by engineering features.

For example, if your data includes longitude and latitude, these columns are treated as numeric, with no special calculations. If location or distance provides signal for your problem, you must engineer a feature that provides that information explicitly.

Some data types that might require feature engineering:

- Longitude/Latitude
- URLs
- IP addresses
- Email addresses
- Phone numbers
- Other geographic codes (for example, postal codes)

Include calculated or aggregated data in a row

AutoML Tables uses only the input data in a single row to predict the target value for that row. If you have calculated or aggregated data from other rows or sources that would be valuable in determining the predicted value for a row, include that data with the source row. Be careful that your new column does not cause target leakage (#target-leakage) or training-serving skew (#training-serving-skew).

For example, if you want to predict next week's demand for a product, you can improve the quality of the prediction by including columns with the following values:

- The total number of items in stock from the same category as the product.
- The average price of items in stock from the same category as the product.
- The number of days before a known holiday when the prediction is requested.
- And so on...

In another example, if you want to predict whether a specific user will buy a product, you can improve the quality of the prediction by including columns with the following values:

- The average historic conversion rate or click-through rate for the specific user.
- How many products are currently in the user's shopping cart.

Represent null values as empty strings

If your data uses special characters or numbers to represent null values, this can present problems for AutoML Tables, because we do not know what they are meant to signify. If you are importing from CSV, use empty strings to represent null values. From BigQuery, use the NULL value.

Avoid missing values where possible

Check your data for missing values, and correct them if possible. Otherwise, you can leave the value blank, if the column is set to be nullable.

Use spaces to separate text

AutoML Tables tokenizes text strings and can derive training signal from individual words. It uses spaces to separate words; words separated by other characters are treated as a single entity.

For example, if you provide the text "red/green/blue", it is not tokenized into "red", "green", and "blue". If those individual words might be important for training the model, you should transform the text to "red green blue" before including it in your training data.

Make sure your categorical features are accurate and clean

Data inconsistencies can cause categories to be incorrectly split. For example, if your data includes "Brown" and "brown", AutoML Tables uses those values as separate categories, when you might have intended them to be the same. Misspellings can have a similar effect. Make sure you remove these kinds of inconsistencies from your categorical data before creating your training data.

Use extra care with imbalanced classes

If you have imbalanced classes (a classification problem with one or more outcomes that is seen rarely), review the following tips.

Provide sufficient training data for the minority class

Having just a few rows of data for one class degrades model quality. If possible, you should provide at least 100 rows of data for every class.

Consider using a manual split

AutoML Tables selects the rows for the test dataset randomly (but deterministically). For imbalanced classes, you could end up with a small number of the minority class in your test dataset, or even none, which causes training to fail.

If you have imbalanced classes, you might want to assign a manual split to make sure enough rows with the minority outcomes are included in every split.

Avoid bias

Make sure that your training data is representative of the entire universe of potential data that you will be making predictions for. For example, if you have customers that live all over the world, you should not use training data from only one country.

Provide enough training data

If you don't provide enough training data (rows), the resulting model might perform poorly. The more features (columns) you use to train your model, the more data (rows) you need to provide. A good goal for classification models is at least 10 times as many rows as you have columns. For regression models, you should provide at least 50 times as many rows as the number of columns.

Your dataset must always include at least 1,000 rows.

Leave all other preprocessing and transformations to AutoML Tables

Unless otherwise noted above, let AutoML Tables do the feature engineering for you. AutoML Tables does best when it has access to your underlying data. See [Data preparation that AutoML Tables does for you](#) (#tables-does).

Data preparation that AutoML Tables does for you

This section lists common requirements for training data that AutoML Tables automatically does for you. You do not need to include these calculations in your training data. In fact, if you perform these transformations yourself and include them in your training data, you might decrease the quality of the resulting model.

The following automatic transformations are applied for each feature column depending on the column type:

Column type	Transformation
Numerical	• The value converted to float32. • The <code>z_score</code> of the value. • A bucket index of the value based on quantiles. Bucket size is 100. • $\log(\text{value}+1)$ when the value is greater than or equal to 0. Otherwise, this transformation is not applied and the value is considered a missing value. • <code>z_score</code> of $\log(\text{value}+1)$ when the value is greater than or equal to 0. Otherwise, this transformation is not applied and the value is considered a missing value. • A boolean value that indicates whether the value is null. • Rows with invalid numerical inputs (for example, a string that can not be parsed to float32) are not included for training and prediction. • Extreme/outlier values are not given any special treatment.
Numerical array	• All transformations for Numerical types applied to the average of the last N items where $N = \{1, 2, 4, 8, \text{all}\}$. So the items most heavily emphasized are the ones towards the end of the array, not the beginning. • The average of empty arrays is treated as zero.
Categorical	• The categorical string as is--no change to case, punctuation, spelling, tense, and so on. • Convert the category name to a dictionary lookup index and generate an embedding for each index. • Categories that appear less than 5 times in the training dataset are treated as the "unknown" category. The "unknown" category gets its own special lookup index and resulting embedding.
Categorical array	• For each element in the array of the last N items where $N = \{1, 2, 4, 8, \text{all}\}$, convert the category name to a

array	dictionary lookup index and generate an embedding for each index. Combine the embedding of all elements into a single embedding using the mean. Empty arrays treated as an embedding of zeroes.
Text	- The text as is--no change to case, punctuation, spelling, tense, and so on. - Tokenize text to words and generate 1-grams and 2-grams from words. Convert each n-gram to a dictionary lookup index and generate an embedding for each index. Combine the embedding of all elements into a single embedding using the mean. <u>Tokenization</u> (https://www.tensorflow.org/beta/tutorials/tensorflow_text/intro#unicodescripttokenizer) is based on unicode script boundaries. - Missing values get their own lookup index and resulting embedding. - Stop-words receive no special treatment and are not removed.
Text array	- Concatenate all text values in the array into a single text value using a space (" ") as a delimiter, and then treat the result as a single text value. Apply the transformations for Text columns. - Empty arrays treated as an embedding of zeroes.
Timestamp	- Apply the transformations for Numerical columns. - Determine the year, month, day, and weekday. Treat each value from the timestamp as a Categorical column. - Invalid numerical values (for example, values that fall outside of a typical timestamp range, or are extreme values) receive no special treatment and are not removed. - Rows with invalid timestamp inputs (for example, an invalid timestamp string) are not included for training and prediction.
Timestamp array	Apply the transformations for Numerical columns to the average of the last N items of the array. N = {1, 2, 4, 8, all}. This means that the items most heavily emphasized are the ones towards the end of the array.
Struct	Struct values are automatically flattened into fields. The flattened fields get treated according to their column type.

Null or missing values

You can choose how null values are handled for your training data by setting that column to be nullable or not in your dataset schema. For more information, see [Creating a Dataset \(/automl-tables/docs/import#create\)](/automl-tables/docs/import#create).

If a null value appears in a non-nullable column, the entire row is excluded from training.

Null values in nullable columns are represented with a special indicator variable that indicates that the value was null or missing. For categorical and text transformations, the indicator results in an embedding.

AutoML Tables treats the following values as null values:

- A BigQuery [NULL value \(/bigquery/docs/reference/standard-sql/data-types\)](/bigquery/docs/reference/standard-sql/data-types).
- NaN or infinite numeric values.
- An empty string. AutoML Tables does not trim spaces from strings. That is, " " is not considered a null value.

- A string that can be converted to NaN or an infinite numeric value (<https://en.cppreference.com/w/c/string/byte/strtof>).
 - For "NaN": ignore case, with an optional plus or minus prepended.
 - For "INF": ignore case, with an optional plus or minus prepended.
- Missing values.

What's next

- Is My Data Any Good? A Pre-ML Checklist
(<http://services.google.com/fh/files/blogs/data-prep-checklist-ml-bd-wp-v2.pdf>)
- Preparing and curating your data for machine learning
(<https://cloud.google.com/blog/products/gcp/preparing-and-curating-your-data-for-machine-learning>)

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2021-03-30 UTC.